

数字系统设计

Digital System Design

----- 逻辑化简与码制

王维东
Weidong Wang

浙江大学信息与电子工程学院
College of Information Science & Electronic Engineering
信息与通信工程系
Zhejiang University

任课教师

- 王维东
- 浙江大学信息与电子工程学院，信电楼 306
- 邮箱: wdwang@zju.edu.cn
- College of Information Science & Electronic Engineering
- Zhejiang University, Hangzhou, 310027
- Tel: 86-571-87953170 (O)
- Mobile: 13605812196

TA:

钟睿, 13687791509; 22331166@zju.edu.cn;

Office Hours : 玉泉信电楼 308 室（可以钉钉、微信或邮件联系）。

Prerequisites 预修课程

- 电子电路基础
- 电子线路
- C 语言

- How to learn this Course?
 - Not only listening, thinking and waiting
 - But Exercise, Simulation, Practice!

课程简介

- 课程代码: **671C0050**
- 参考书
 - 阎石, 《数字电子技术基础》, 第 6 版, 高等教育出版社, 2016.
 - 王金明著, 《数字系统设计与 Verilog HDL 》, 电子工业出版社, 第 8 版
 - 《数字系统设计 - 补充讲义》, 浙江大学自编讲义
 - Stanford 大学 108A 课程 notes.
 - R.H.Katz, G.Borriello, *Contemporary Logic Design*, second edition, 电子工业出版社, 2005.
 - M.M.Mano, 数字设计 (第四版), 电子工业出版社, 2010.
 - » <http://www.prenhall.com/mano>

Other Course Info

- **Website:**

- <http://mypage.zju.edu.cn/wdwd/> 教学工作 /
- **学在浙大：数字系统设计**
- Check frequently

- **答疑**

- 玉泉信电楼 308 室 / 周一周四下午 2:30-5:30
- 上课课间、课后均可
- Email , 钉钉 / 微信群 / 数字系统设计, 短信均可

Grading (考核)

Final grades will be computed approximately as follows:

- 平时 (含课程作业、期中考试、小测验、 **Project**、出勤等) **40%**
 - 期中考试 **10%**
 - Homework Sets **15% (12~20)%**
 - **Project** **5% (5~7)%**
 - Class Room Check+ 小测验等 **10% (7~10)%**
 - 作业每周一上交, **截止期为课后一周内**有效。
- **Project**
 - Project-1 必做 (1 members)
 - Project-2 可选(成绩加分 1%~5%, 但不超过平时成绩范围)
- **Finial Exam** 期末闭卷考试 - **60%**

授课时间和地点:

- 2025 年春夏学期,
 - 周一上午, 第 1~2 节 (8:00-9:35)
 - 星期四上午, 第 6~7 节 (13:25-13:00)
- 地点: 紫金港西 2-415(多)
- <http://mypage.zju.edu.cn/wdwd/> 教学工作 /
- 学在浙大 / 数字系统设计
- 钉钉 / 微信 / 数字系统设计 - 答疑

课程结构

- 数字理论知识 (必备)
 - 数字系统和编码、逻辑代数、门电路
- 数字电路分析与设计
 - 组合逻辑电路
 - 触发器、半导体存贮器、可编程器件
 - 时序逻辑电路
- 脉冲电路与接口
- 控制器与数字系统
 - 状态机
 - 控制器
 - 微码控制器
 - 测试和验证
- 微处理器简介
 - 指令集
 - **4 位 CPU** 介绍

---- 函数化简 & 数制码制

逻辑函数的化简法

- 公式化简法

- 反复应用基本公式和常用公式，消去多余的乘积项和多余的因子。

- 卡诺图化简法

- 将逻辑函数的最小项之和的以图形的方式表示出来。

组合电路逻辑关系的图形表示法可以追溯到：

- 英国逻辑学家约翰·维恩（John Venn）1881年发明的在集合论中处理集合间逻辑关系的文氏图（Venn diagram）
- 赫尔姆·哈斯（Helmut Hasse）有效地利用 Vogt 在 1895 年用过的哈斯图（Hasse diagram）来表示序理论中的有限偏序集
- 爱德华·维奇（Edward W. Veitch）在 1952 年将维恩图中的圆形改画成矩形而发明了维奇图（Veitch diagram）。

美国贝尔实验室的电信工程师莫里斯·卡诺（Maurice Karnaugh）在 1953 年根据维奇图改进的卡诺图（Karnaugh map）或 K 图（K-map）在数字逻辑、故障诊断等许多领域中应用广泛。

逻辑函数的化简法

- 逻辑函数的最简形式

最简与或

----- 包含的乘积项已经最少，每个乘积项的因子也最少，称为最简的与 - 或逻辑式。

$$Y_1 = ABC + B'C + ACD$$

$$Y_2 = AC + B'C$$

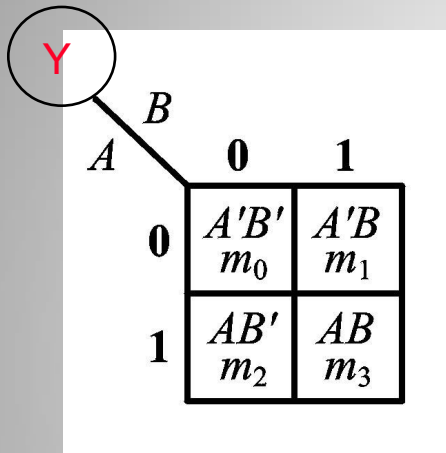
卡诺图化简法

逻辑函数的卡诺图表示法

- **实质**：将逻辑函数的最小项之和的以图形的方式表示出来
- 以 2^n 个小方块分别代表 n 变量的所有最小项，并将它们排列成矩阵，而且使**几何位置相邻**的两个最小项在**逻辑上也是相邻的**（只有一个变量不同），就得到表示 n 变量全部最小项的卡诺图。

表示最小项的卡诺图

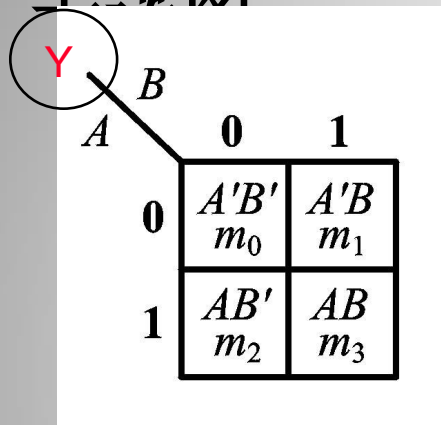
- 二变量卡诺图



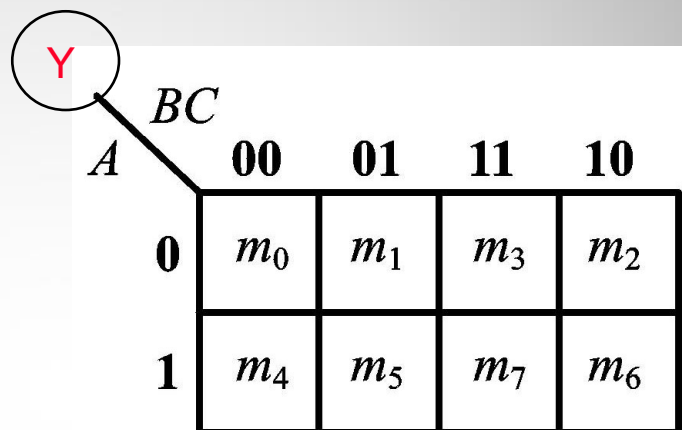
表示最小项的卡诺图

- 二变量卡诺图

卡诺图

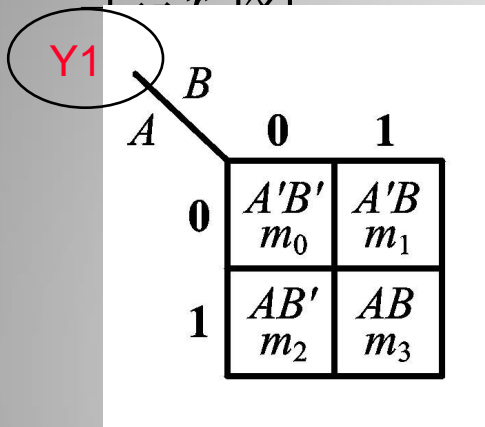


三变量的

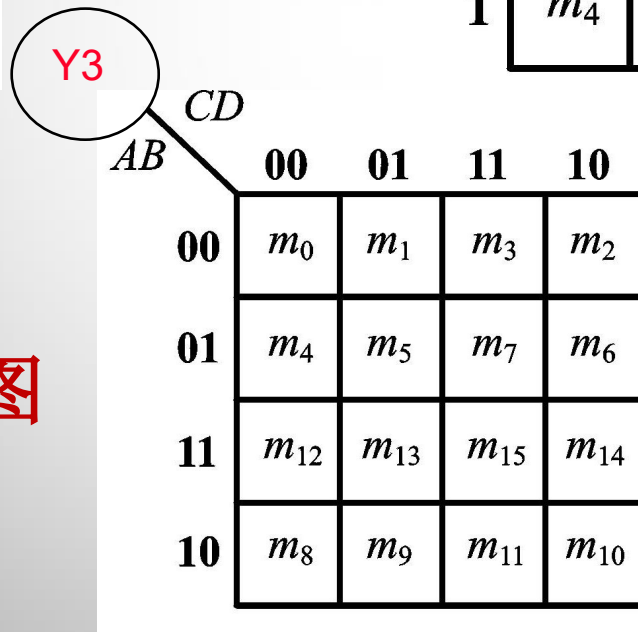
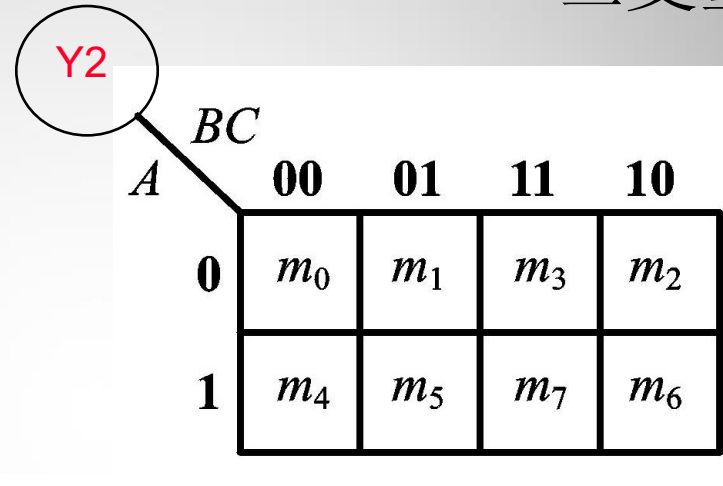


表示最小项的卡诺图

- 二变量卡诺图



三变量的



4 变量的卡诺图

- 五变量的卡诺图

Y

		<i>CDE</i>							
		<i>AB</i>	000	001	011	010	110	111	101
	00	m_0	m_1	m_3	m_2	m_6	m_7	m_5	m_4
	01	m_8	m_9	m_{11}	m_{10}	m_{14}	m_{15}	m_{13}	m_{12}
	11	m_{24}	m_{25}	m_{27}	m_{26}	m_{30}	m_{31}	m_{29}	m_{28}
	10	m_{16}	m_{17}	m_{19}	m_{18}	m_{22}	m_{23}	m_{21}	m_{20}

用卡诺图表示逻辑函数

1. 将函数表示为最小项之和的形式 $\sum m_i$ 。
2. 在卡诺图上与这些最小项对应的位置上填入 1，其余地方填 0。

用卡诺图表示逻辑函数

例:

$$\begin{aligned} Y(A, B, C, D) &= A'B'C'D + A'BD' + AB' \\ &= A'B'C'D + (C + C')A'BD' + AB'[(CD)' + C'D + CD' + CD] \\ &= \sum m(1, 4, 6, 8, 9, 10, 11, 15) \end{aligned}$$

用卡诺图表示逻辑函数

Y

CD

AB

	00	01	11	10
00	0	1	0	0
01	1	0	0	1
11	0	0	1	0
10	1	1	1	1

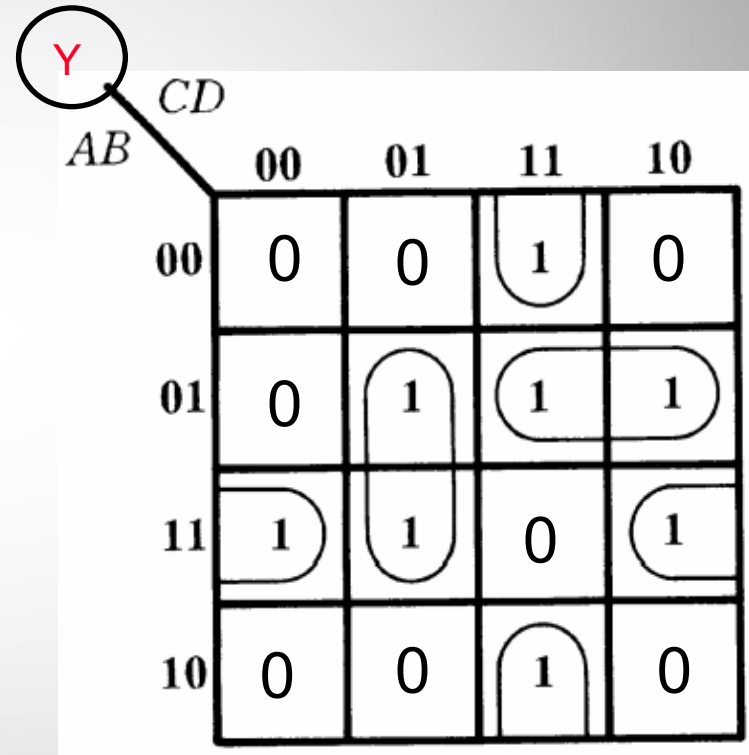
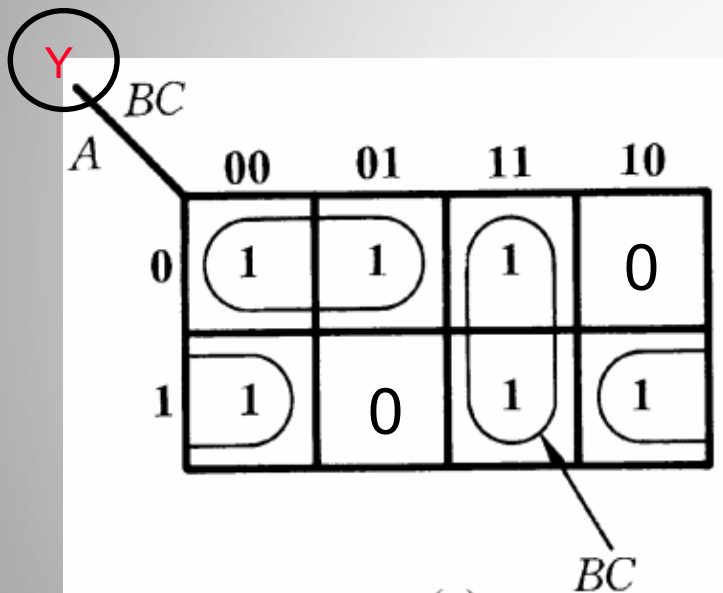
用卡诺图化简函数

- **依据：**具有相邻性的最小项可合并，消去不同因子。
- 在卡诺图中，最小项的相邻性可以从图形中直观地反映出来。

- **合并最小项的原则：**

- 两个**相邻**最小项可合并为一项，消去一对因子
- 四个排成矩形的**相邻**最小项可合并为一项，消去两对因子
- 八个**相邻**最小项可合并为一项，消去三对因子

两个相邻最小项可合并为一项，
消去一对因子



用卡诺图化简函数

- 化简步骤:

- 1)----- 用卡诺图表示逻辑函数

- 2)----- 找出可合并的最小项

- 3)----- 化简后的乘积项相加

(项数最少, 每项因子最少)

卡诺图化简的原则

- 化简后的乘积项应包含函数式的所有最小项，
即覆盖图中所有的 **1**。
- 乘积项的数目最少，即圈成的矩形最少。
- 每个乘积项因子最少，即圈成的矩形最大。

例:

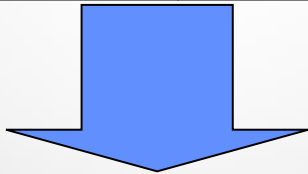
$$Y(A,B,C) = AC' + A'C + B'C + BC'$$

Y		BC			
A		00	01	1 1	1 0
0					
1					

例:

$$Y(A,B,C) = AC' + A'C + B'C + BC'$$

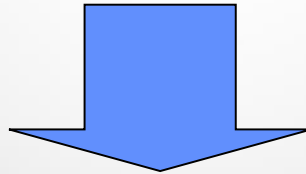
Y A	BC	00	01	1 1	1 0
0		0	1	1	1
1		1	1	0	1


$$AB' + A'C + BC'$$

例:

$$Y(A,B,C) = AC' + A'C + B'C + BC'$$

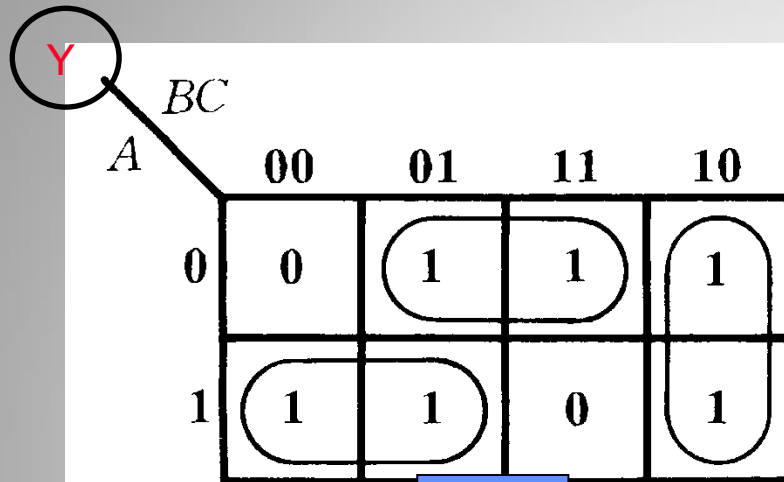
Y A		BC			
		00	01	1 1	1 0
0		0	1	1	1
1		1	1	0	1



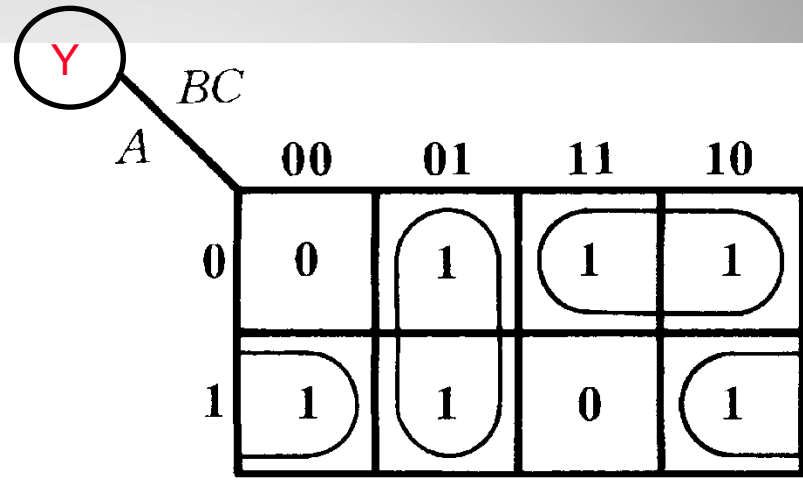
$$AC' + A'B + B'C$$

例:

$$Y(A,B,C) = AC' + A'C + B'C + BC'$$



$$AB' + A'C + BC'$$



$$AC' + A'B + B'C$$

化简结果不唯一

例:

$$Y = ABC + ABD + AC'D + C' \cdot D' + AB'C + A'CD'$$

Y	CD			
	00	01	11	10
AB	00			
	01			
	11			
	10			

例:

$$Y = ABC + ABD + AC'D + C' \cdot D' + AB'C + A'CD'$$

Y

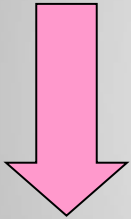
AB	CD			
	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	1	1	1
10	1	1	1	1

$$A + D'$$

具有无关项的逻辑函数及其化简

约束项、任意项和逻辑函数式中的无关项

- 约束项
- 任意项



在逻辑函数中，对输入变量取值的限制，在这些取值下为 1 的最小项称为约束项

在输入变量某些取值下，函数值为 1 或为 0 不影响逻辑电路的功能，在这些取值下为 1 的最小项称为任意项

- 逻辑函数中的无关项：约束项和任意项可以写入函数式，也可不包含在函数式中，因此统称为无关项。

无关项在化简逻辑函数中的应用

- 合理地利用无关项，可得更简单的化简结果。
- 加入（或去掉）无关项，应使化简后的项数最少，每项因子最少……

从卡诺图上直观地看，加入无关项的目的是为矩形圈最大，矩形组合数最少。

例： $Y = A'B'C'D + A'BCD + AB'C' \cdot D'$

给定约束条件为：

$$A'B'CD + A'BC'D + ABC' \cdot D' + AB' \cdot C'D + ABCD + ABCD' + AB'CD' = 0$$

Y

	CD			
AB	00	01	11	10
00		1		
01			1	
11				
10	1			

例 : $Y = A'B'C'D + A'BCD + AB'C' \cdot D'$

给定约束条件为:

$$A'B'CD + A'BC'D + ABC' \cdot D' + AB' \cdot C'D + ABCD + ABCD' + AB'CD' = 0$$

Y		CD			
		00	01	11	10
AB	00	0	1	x	0
	01	0	x	1	0
	11	x	0	x	x
	10	1	x	0	x

例 : $Y = A'B'C'D + A'BCD + AB'C' \cdot D'$

给定约束条件为:

$$A'B'CD + A'BC'D + ABC' \cdot D' + AB' \cdot C'D + ABCD + ABCD' + AB'CD' = 0$$

Y

		CD			
		00	01	11	10
AB	00	0	1	x	0
	01	0	x	1	0
	11	x	0	x	x
	10	1	x	0	x

$A'D$

AD'

例:

$$Y(A, B, C, D) = \sum m(2, 4, 6, 8)$$

$$\text{约束条项: } m_5 + m_{10} + m_{11} + m_{12} + m_{13} + m_{14} + m_{15} = 0$$

Y

	CD			
AB	00	01	11	10
00	0	0	0	1
01	1	x	0	1
11	x	x	x	x
10	1	0	x	x

$$Y = AD' + BD' + CD'$$

原码、反码与补码

- 原码

- 符号位

- 0 为 +

- 1 为 -

- 00011010

- 10101101

- 1000

- 反码

- $N=N$ 正数

- $N=(2^n-1)-N$
负数

- 每位取
反, 1 的补
码

- 00011010

- 11010010

- 1111

- 补码

- $N=N$ 正数

- $N=2^n-N$ 负数

- 反码加 1

- 00011010

- 11010011

- 1000

数制与编码（复习）

NUMBER SYSTEMS AND CODES

- 二进 - 十进制转换 BINARY-TO-DECIMAL CONVERSIONS

➤ 例 1

$$\begin{array}{cccccc} 1 & 1 & 0 & 1 & 1_2 \\ 2^4 & + & 2^3 & + & 0 & + & 2^1 & + & 2^0 & = & 16 & + & 8 & + & 2 & + & 1 \\ & & & & & & & & & = & 27_{10} \end{array}$$

➤ 例 2

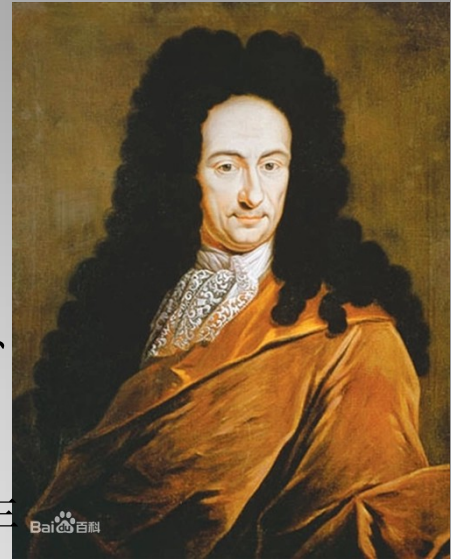
$$\begin{array}{cccccccc} 1 & 0 & 1 & 1 & 0 & 1 & 0 & 1_2 = \\ 2^7 & + & 0 & + & 2^5 & + & 2^4 & + & 0 & + & 2^2 & + & 0 & + & 2^0 & = & 181_{10} \end{array}$$

二进制和中国古代八卦图

- 莱布尼茨自己的说法

- 18 世纪初，莱布尼茨收到耶稣会士白晋所寄的伏羲八卦图，正式研究八卦符号，以此完善了自己的二进制体系，然后写了论文《二进位算术的阐述—关于只用 0 和 1 兼论其用处及伏羲氏所用数字的意义》，发表在 1705 年法国《皇家科学院院刊》上。

- 二进制就是这样诞生的。



戈特弗里德·威廉·莱布尼茨（**Gottfried Wilhelm**

Leibniz，1646 年 7 月 1 日—1716 年 11 月 14 日），德国哲学家、数学家，是历史上少见的通才，被誉为十七世纪的亚里士多德。

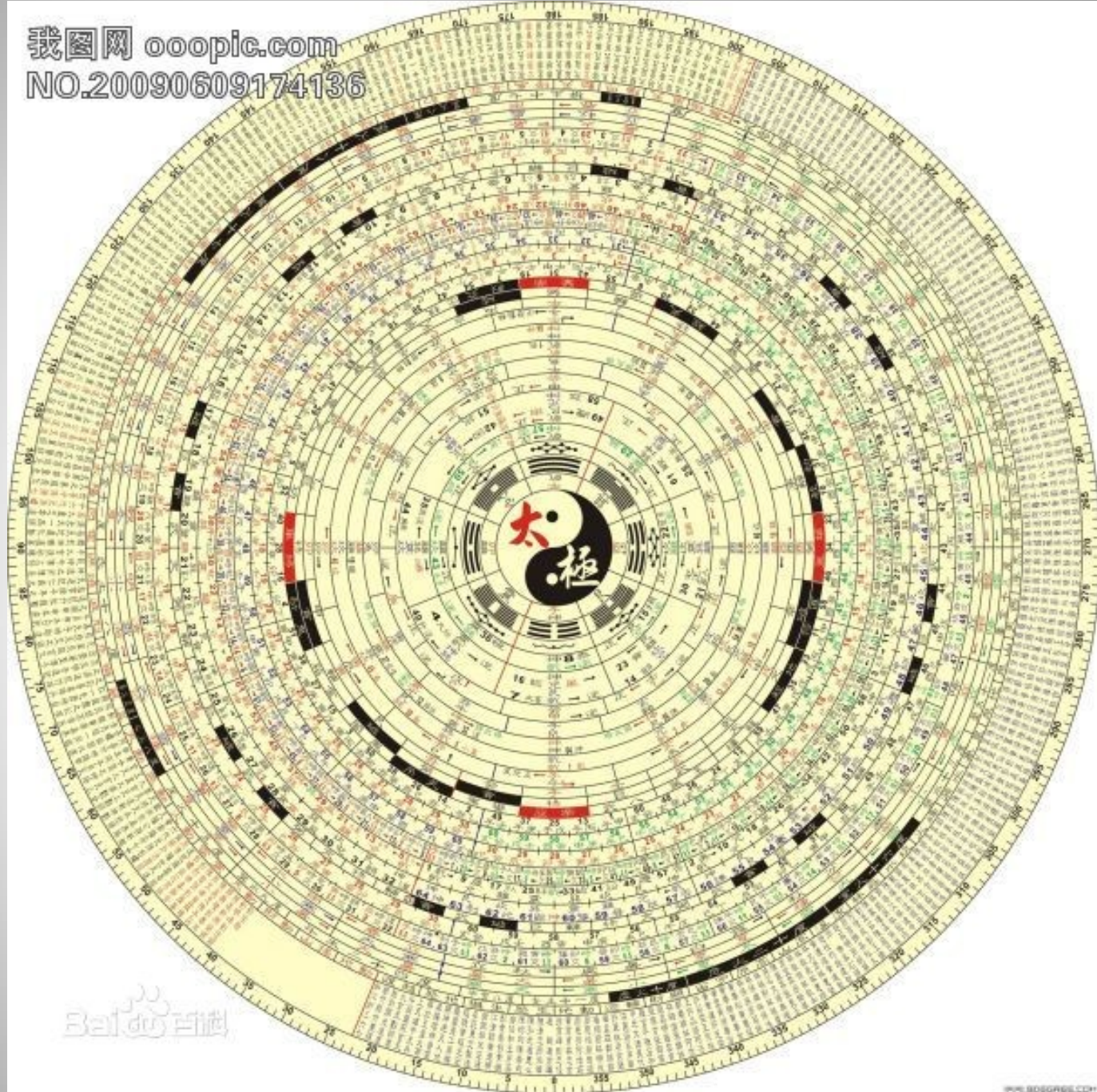


八卦图衍生自汉族古代的《河图》与《洛书》，传为伏羲所作。其中《河图》演化为先天八卦，《洛书》演化为后天八卦。八卦各有三爻，“乾、坤、震、巽、坎、离、艮、兑”分立八方，象征“天、地、雷、风、水、火、山、泽”八种性质与自然现象，象征世界的变化与循环，分类方法如同五行，世间万物皆可分类归至八卦之中。

太极八卦

我图网 ooopic.com
NO.20090609174136

- **太极八卦**即是阐明宇宙从无极而太极，以至万物化生的过程。
- **其中的太极**即为天地未开、混沌未分阴阳之前的状态。
- **两仪**即为太极的阴、阳二仪。
- 《系辞》又说：“两仪生**四象**，四象生**八卦**”。



数制

$$\begin{aligned} 45_{10} &= 32 + 8 + 4 + 1 = 2^5 + 0 + 2^3 + 2^2 + 0 + 2^0 \\ &= 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1_2 \end{aligned}$$

$$\begin{aligned} 76_{10} &= 64 + 8 + 4 = 2^6 + 0 + 0 + 2^3 + 2^2 + 0 + 0 \\ &= 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0_2 \end{aligned}$$

十六进制

HEXADECIMAL NUMBER SYSTEM

- 16 位数字

- 0,1,2,3,4,5,6,7,8,9,

- A=10,

- B=11,

- C=12,

- D=13,

- E=14,

- F=15

Hexadecimal	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

16^4	16^3	16^2	16^1	16^0	16^{-1}	16^{-2}	16^{-3}	16^{-4}
--------	--------	--------	--------	--------	-----------	-----------	-----------	-----------

Hexadecimal point

十六进制转换

- Hex-to-Decimal Conversion

$$\begin{aligned} 356_{16} &= 3 \times 16^2 + 5 \times 16^1 + 6 \times 16^0 \\ &= 768 + 80 + 6 \\ &= 854_{10} \end{aligned}$$

$$\begin{aligned} 2AF_{16} &= 2 \times 16^2 + 10 \times 16^1 + 15 \times 16^0 \\ &= 512 + 160 + 15 \\ &= 687_{10} \end{aligned}$$

十六进制转换

- Decimal-to-Hex Conversion

(a) Convert 423_{10} to hex.

Solution

$$\begin{array}{l} \frac{423}{16} = 26 + \text{remainder of } 7 \\ \downarrow \\ \frac{26}{16} = 1 + \text{remainder of } 10 \\ \downarrow \\ \frac{1}{16} = 0 + \text{remainder of } 1 \end{array}$$

$423_{10} = 1A7_{16}$

十六进制转换

Hex-to-Binary Conversion

Bin

$$\begin{aligned} 9F2_{16} &= \begin{array}{ccccccc} & 9 & & F & & 2 & \\ & \downarrow & & \downarrow & & \downarrow & \\ & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 0 \end{array} \\ &= 100111110010_2 \end{aligned}$$

$$\begin{aligned} 1110100110_2 &= \underbrace{0011}_3 \underbrace{1010}_A \underbrace{0110}_6 \\ &= 3A6_{16} \end{aligned}$$

十六进制

- **Counting in Hexadecimal**

(a) 38, 39, 3A, 3B, 3C, 3D, 3E, 3F, 40, 41, 42

(b) 6F8, 6F9, 6FA, 6FB, 6FC, 6FD, 6FE, 6FF, 700

- With N hex digit positions, we can count from decimal 0 to $16^N - 1$, for a total of 16^N different values.
- For example, with three hex digits,
 - we can count from 000_{16} to FFF_{16} ,
 - which is 0_{10} to 4095_{10} ,
 - for a total of $4096 = 16^3$ different values.

码制

When numbers, letters, or words are represented by a special group of symbols, we say that they are being encoded, and the group of symbols is called a code.

- **BCD CODE**

- a decimal number is represented by its equivalent binary number
- straight binary coding

- **Binary-Coded-Decimal Code**

- Digital systems all use some form of binary numbers for their internal operation, but the external world is decimal in nature.
- each digit of the decimal number by a four-bit binary number.
- The BCD code does not use the numbers 1010, 1011, 1100, 1101, 1110, and 1111.

9	4	3	(decimal)
↓	↓	↓	
1001	0100	0011	(BCD)

BCD CODE

Comparison of BCD and Binary

$$137_{10} = 10001001_2 \quad (\text{binary})$$

$$137_{10} = 0001 \ 0011 \ 0111 \quad (\text{BCD})$$

码制

• THE GRAY CODE

- Digital systems operate at very fast speeds and respond to changes that occur in the digital inputs.
- When **multiple** input conditions are **changing** at the same time, the situation can be misinterpreted 误解 and cause an erroneous reaction.
- In order to reduce the likelihood of a digital circuit misinterpreting a changing input, the **Gray code** has been developed as a way to represent a sequence of numbers.

典型格雷码（ Primary GrayCode ） 的权威描述是 Frank Gray 的美国 "PulseCode Communication[8]". 该申请于 1947 年 11 月 13 日，申请号为 785697; 1953 年 3 月 17 日获得批准，号为 2632058, 权归贝尔电话实验室所有。
Gray Code 因 Frank Gray 命名，绝不能写成 Grey Code; 可写为 Gray code, 但不能写成 gray code.

Gray code

- the Gray code
 - **only one bit** ever changes between two successive numbers in the **sequence**.
- To convert binary to Gray
 - the most significant bit use as the Gray MSB.
 - compare the MSB binary with the next binary bit (B1).
 - » If they are the same, then $G1=0$.
 - » If they are different, then $G1=1$.
 - $G0$ can be found by comparing B1 with B0.

B ₂	B ₁	B ₀	G ₂	G ₁	G ₀
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	0	1
1	1	1	1	0	0

PUTTING IT ALL TOGETHER

Decimal	Binary	Hexadecimal	BCD	GRAY
0	0	0	0000	0000
1	1	1	0001	0001
2	10	2	0010	0011
3	11	3	0011	0010
4	100	4	0100	0110
5	101	5	0101	0111
6	110	6	0110	0101
7	111	7	0111	0100
8	1000	8	1000	1100
9	1001	9	1001	1101
10	1010	A	0001 0000	1111
11	1011	B	0001 0001	1110
12	1100	C	0001 0010	1010
13	1101	D	0001 0011	1011
14	1110	E	0001 0100	1001
15	1111	F	0001 0101	1000

THE BYTE, NIBBLE, AND WORD

- **Bytes**

- binary data and information in groups of eight bits
- A byte always consists of eight bits
- it can represent any of numerous types of data or information

- **Nibbles(点心 - 半字节)**

- Binary numbers are often broken down into groups of four bits,
- Because it is half as big as a byte, it was named a **nibble**.

- **Words**

- The size of the word **depends** on the size of the data pathway in the system that uses the information
 - » the computer in your microwave oven can probably handle only one byte at a time. It has a word size of eight bits.
 - » the personal computer on your desk can handle eight bytes at a time, so it has a word size of 64 bits.

ALPHANUMERIC CODES

- A complete alphanumeric code would include
 - the 26 lowercase letters,
 - 26 uppercase letters,
 - 10 numeric digits,
 - 7 punctuation marks,
 - and anywhere from 20 to 40 other characters,
 - » such as , /, #, %, *,
 - and so on.
- **ASCII Code**
 - American Standard Code for Information Interchange (ASCII).

ASCII Code

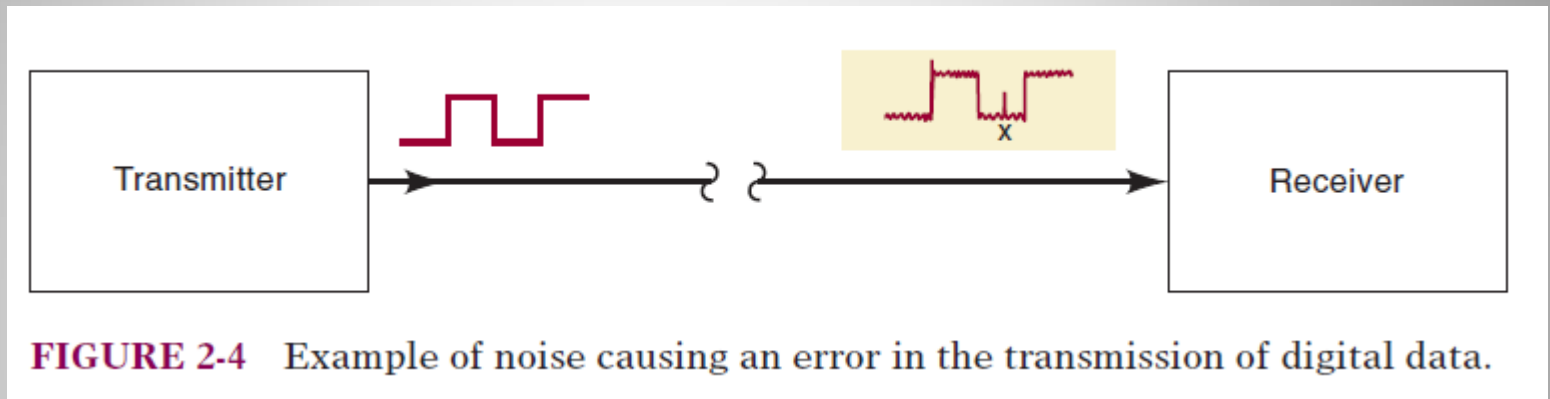
- The ASCII code

- pronounced “askee”
- is a seven-bit code
- so it has $2^7=128$ possible code groups

TABLE 2-4 Standard ASCII codes.

Character	HEX	Decimal	Character	HEX	Decimal	Character	HEX	Decimal	Character	HEX	Decimal
NUL (null)	0	0	Space	20	32	@	40	64	`	60	96
Start Heading	1	1	!	21	33	A	41	65	a	61	97
Start Text	2	2	"	22	34	B	42	66	b	62	98
End Text	3	3	#	23	35	C	43	67	c	63	99
End Transmit.	4	4	\$	24	36	D	44	68	d	64	100
Enquiry	5	5	%	25	37	E	45	69	e	65	101
Acknowledge	6	6	&	26	38	F	46	70	f	66	102
Bell	7	7	'	27	39	G	47	71	g	67	103
Backspace	8	8	(	28	40	H	48	72	h	68	104
Horiz. Tab	9	9	)	29	41	I	49	73	i	69	105
Line Feed	A	10	*	2A	42	J	4A	74	j	6A	106
Vert. Tab	B	11	+	2B	43	K	4B	75	k	6B	107
Form Feed	C	12	,	2C	44	L	4C	76	l	6C	108
Carriage Return	D	13	-	2D	45	M	4D	77	m	6D	109
Shift Out	E	14	.	2E	46	N	4E	78	n	6E	110
Shift In	F	15	/	2F	47	O	4F	79	o	6F	111
Data Link Esc	10	16	0	30	48	P	50	80	p	70	112
Direct Control 1	11	17	1	31	49	Q	51	81	q	71	113
Direct Control 2	12	18	2	32	50	R	52	82	r	72	114
Direct Control 3	13	19	3	33	51	S	53	83	s	73	115
Direct Control 4	14	20	4	34	52	T	54	84	t	74	116
Negative ACK	15	21	5	35	53	U	55	85	u	75	117
Synch Idle	16	22	6	36	54	V	56	86	v	76	118
End Trans Block	17	23	7	37	55	W	57	87	w	77	119
Cancel	18	24	8	38	56	X	58	88	x	78	120
End of Medium	19	25	9	39	57	Y	59	89	y	79	121
Substitutue	1A	26	:	3A	58	Z	5A	90	z	7A	122
Escape	1B	27	;	3B	59	[	5B	91	{	7B	123
Form separator	1C	28	<	3C	60	\	5C	92		7C	124
Group separator	1D	29	=	3D	61	]	5D	93	}	7D	125
Record Separator	1E	30	>	3E	62	^	5E	94	~	7E	126
Unit Separator	1F	31	?	3F	63	_	5F	95	Delete	7F	127

PARITY METHOD FOR ERROR DETECTION



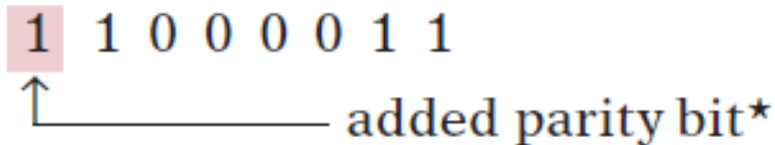
ERROR DETECTION

- **Parity Bit 校验位**

- an extra bit that is attached to a code group
- The parity bit is made either 0 or 1,
- depending on the number of 1s that are contained in the code group

- ***even-parity* method 偶校验**

- total number of 1s in the code group (including the parity bit) is an *even* number.



1 1 0 0 0 0 1 1
↑ added parity bit*

- The *new* code group, *including the parity bit*
- add a parity bit of 1 to make the total number of 1s an even number

Parity Bit

- The **odd-parity** method 奇校验
 - the total number of 1s (including the parity bit) is an *odd* number
- the parity bit becomes an actual part of the code word
 - Regardless of whether even parity or odd parity is used,
- For example
 - adding a parity bit to the **seven**-bit ASCII code produces an **eight**-bit code

parity bit

- The parity bit is issued to detect any **single-bit errors** that occur during the transmission of a code from one location to another.

- For example

- Suppose that the character “A” is being transmitted and odd parity is being used.
- The transmitted code would be

1 1 0 0 0 0 0 1

- When the receiver circuit receives this code, it will check that the code contains an odd number of 1s (including the parity bit).
- If so, the receiver will assume that the code has been correctly received.

parity bit

- For example

- The transmitted code would be

1 1 0 0 0 0 0 1

- suppose that because of some noise or malfunction the receiver actually receives the following code:

1 1 0 0 0 0 0 0

- The receiver will find that this code has an even number of 1s.
 - This tells the receiver that there must be an error in the code because presumably the transmitter and receiver have agreed to use odd parity.
 - It should be apparent that this parity method would not work if *two* bits were in error,

parity bit

- For example

attached even-parity bits
↓

H =	0	1	0	0	1	0	0	0
E =	1	1	0	0	0	1	0	1
L =	1	1	0	0	1	1	0	0
L =	1	1	0	0	1	1	0	0
O =	1	1	0	0	1	1	1	1

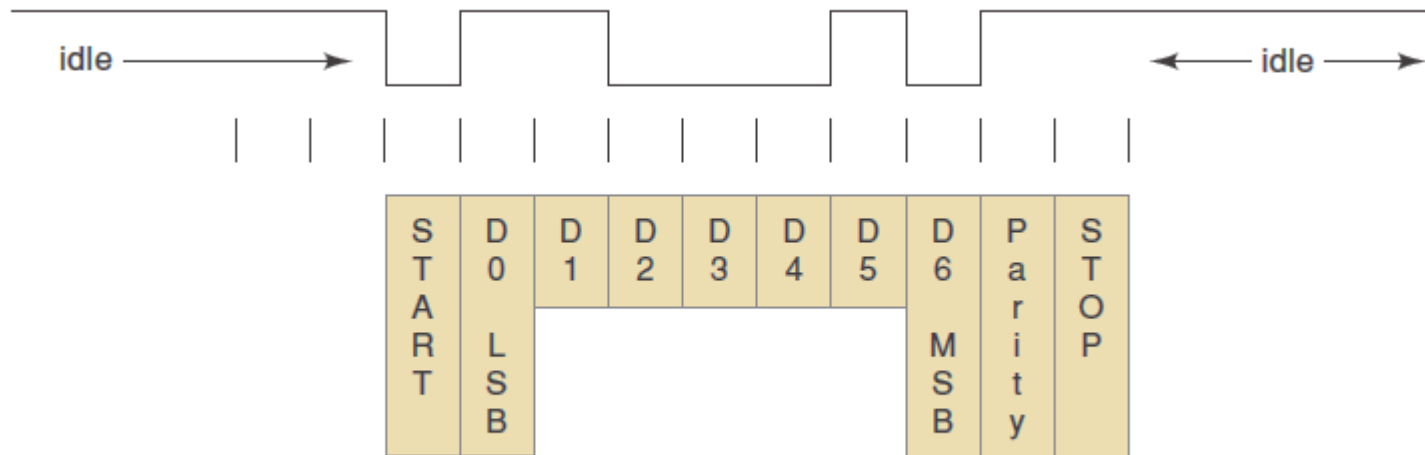


FIGURE 2-5 Asynchronous serial data with even parity.

复习（课本）

- 常见十进制代码
 - 8421 码
 - 余 3 码
 - 2421 码
 - 5211 码
 - 余 3 循环码
- 有什么特点？
- 有什么用途？

课后作业

- 调查：
 - 国际上的数字相关集成电路公司有哪些芯片？
 - » TI, Philips, Toshiba, Fairchild, Motorola.....
 - 图书馆资源：电子器件天地
 - » http://10.15.61.7:8080/poweb/s_search.jsp?searchFiled=5&KWord=tjn&sClass=0&Submit=%BC%EC+%CB%F7
- 作业：
 - 作业见 “学在浙大 / 数字系统设计”
- 阅读
 - 组合电路

什么是电子设计竞赛？什么是 SRTP？

- 了解规则
- 了解课题
- 组队
- 联系指导教师
- 申报
- go